VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Employee Attendance Analyzer

A9511- Python Programming Laboratory

Submitted by
S. Harini
25881A0580
B. Jashwanth
25881A0583

Course Facilitator

P.S. Madhavi
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

Academic Year 2026-27

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

Vibrant Vardhaman
25 Years of

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Employee Attendance Analyzer"** is carried out by **S. Harini** and **B. Jashwanth** with Roll Numbers **25881A0580** and **25881A0583** towards **A9511 - Python Programming Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course

P.S. Madhavi
Assistant Professor, CSE

Signature of the HOD
Dr. Y. Vijayalata Professor,
Head of the Department & Dean (Student Mentoring) Department Computer Science and Engineering,
Vardhaman College of Engineering, Hyderabad

Contents

1. Introduction 2
2. Literature Review 2
3. Objectives 3
4. Methodology 4
4.1 Architectural Diagram 4
4.2 Hardware and Software Requirements 4
4.3 Working Principle 5
4.4 Software Implementation 8
5. Results and Discussion ..... .25
6. Mapping POs, PSOs and SDGs ..... .29
6.1 Program Outcomes (POs) Mapping ..... .29
6.2 Sustainable Development Goals (SDGs) Mapping ..... .29
6.3 Program Specific Outcomes(PSOs) Mapping ..... .30
7. Conclusion ..... .31
Bibliography ..... .32

Abstract

The Employee Attendance Analyzer is a desktop application developed in Python using the Tkinter/ttk toolkit for the graphical interface and the built-in SQLite database (sqlite3) for storage. It allows an administrator to register employees with a unique whole-number ID, mark each employee Present or Absent for any date with a mandatory reason for every absence, correct or delete saved records, and analyse the data through a dashboard, a date-wise search, a weekly report, a monthly report and a ranking of employees by attendance percentage.

The application protects data quality at the point of entry: IDs must be positive whole numbers, one ID can belong to only one employee name, only one record is allowed per employee per date, dates must follow the YYYY-MM-DD format, and every SQL statement uses parameters. A start-up routine upgrades older databases automatically by adding the employee_id column and assigning numeric IDs to legacy employees. Every report can be exported to a CSV file that opens correctly in Excel, and a built-in test-data generator fills the previous two months with weekday records so that the reports can be demonstrated immediately. The complete system is a single Python file that depends only on the standard library.

Keywords: Attendance Management, Python, Tkinter, SQLite, Data Analysis, Reports, CSV Export, Desktop Application, Data Validation

1 Introduction

Attendance is one of the most basic records an organisation keeps. It influences payroll, leave planning, performance reviews and compliance with labour policies. In many small offices, laboratories and institutions the attendance register is still maintained on paper or in loosely structured spreadsheets, which makes it slow to total, easy to alter by mistake and difficult to analyse over a week or a month.

Typical problems of manual registers include duplicate entries for the same person and day, misspelt names that split one employee into two, missing reasons for absence, and the effort needed to count present and absent days at the end of every period. A system that validates every entry and calculates the totals automatically removes these problems and gives managers reliable numbers whenever they need them.

The Employee Attendance Analyzer addresses this need with a clean desktop interface built from a dark sidebar and a light content area. Five screens are available from the sidebar: Dashboard, Mark Attendance, Date-wise Search, Weekly Report and Monthly Report. Attendance is stored in a local SQLite file named attendance.db, so no database server has to be installed and the data stays on the user's own computer.

Besides recording attendance, the project demonstrates several core software-engineering ideas in a small, readable code base: layered design (interface, logic and data), input validation, parameterised SQL queries, aggregation with GROUP BY, safe schema migration, and exporting data for use in other tools.

2 Literature Review

Attendance tracking has moved through several generations. Paper registers and spreadsheets are cheap and familiar, but they depend on manual totalling and offer no built-in protection against duplicate or inconsistent entries. Card, biometric and face-recognition systems capture the time of arrival automatically, yet they need dedicated hardware and are usually bundled inside larger human-resource platforms that small teams may find costly or complicated.

Cloud-based and web attendance tools provide access from anywhere, but they require an internet connection, user accounts and ongoing subscriptions. For a single administrator or a small department, a lightweight desktop tool with local storage is often sufficient and easier to keep private. Python with Tkinter is a common choice for such tools because the toolkit ships with the language, runs on Windows, Linux and macOS, and provides ready-made widgets such as the Treeview table and the Combobox.

On the data side, SQLite is a self-contained, serverless relational database that is included in Python's standard library. It supports SQL aggregation, transactions and constraints such as primary keys, which are exactly the features needed to store attendance records and to compute present and absent counts per employee, per week or per month. Database textbooks also stress validating data before it is stored and using parameterised queries, both of which are applied throughout this project.

3 Objectives

Objective 1: Reliable attendance recording

To build a desktop application in which attendance for any employee and date can be saved, updated and deleted, with strict validation of employee IDs, names, dates, status and the reason for absence, and with protection against duplicate records.

Objective 2: Meaningful analysis and reporting

To provide a dashboard with key totals, a date-wise search, weekly and monthly reports and an attendance ranking, all calculated from the stored data using SQL aggregation, and to let every report be exported to CSV.

Objective 3: Simple, portable and maintainable software

To deliver the system as a single Python file with no external dependencies, using a local SQLite database that upgrades itself when an older database is opened, so that it is easy to install, demonstrate and extend.

4 Methodology

4.1 Architectural Diagram

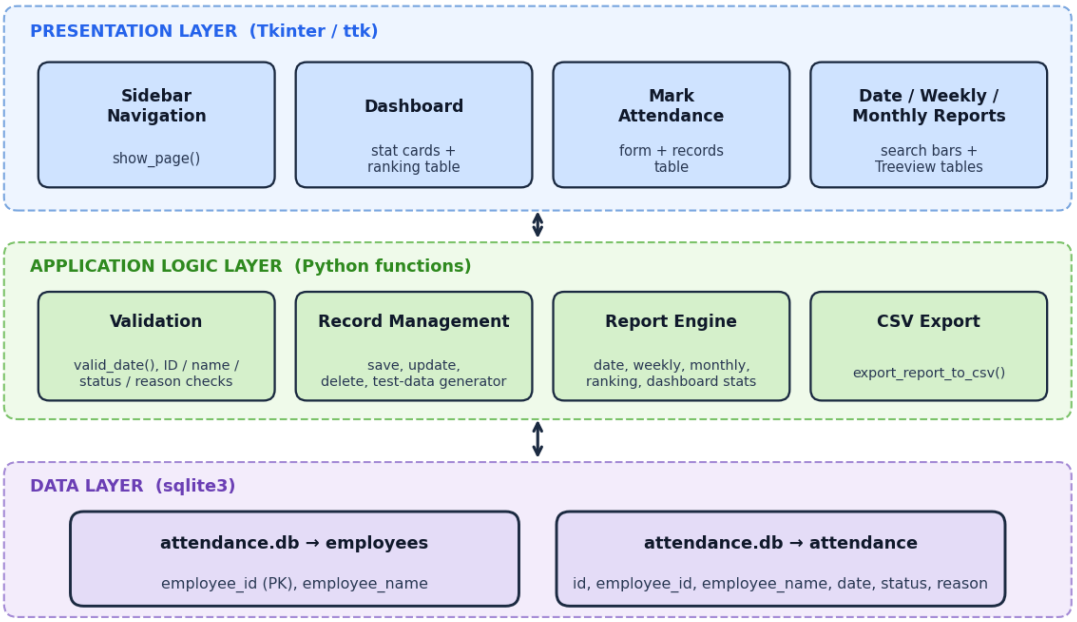


Figure 1: Layered architecture of the Employee Attendance Analyzer

The application follows a simple three-layer structure. The presentation layer is built with Tkinter and ttk. A dark sidebar holds the navigation buttons, and a content area shows one page at a time; the function show_page() hides every page, displays the selected one and refreshes its data. Each page is a Frame containing labels, entry fields, buttons and Treeview tables.

The application logic layer is a set of Python functions. valid_date() checks dates, save_attendance() applies all validation rules before inserting or updating a record, the load_* functions build the date, weekly, monthly and ranking reports, refresh_dashboard() calculates the summary figures, and export_report_to_csv() writes the visible table to a file chosen by the user.

The data layer is the SQLite file attendance.db with two tables. The employees table stores each employee_id (primary key) and employee_name. The attendance table stores a record id, the employee_id, the employee_name, the attendance_date, the status (Present or Absent) and the reason. Every query is executed through the connect_db() helper with ? placeholders for all user-supplied values.

4.2 Hardware and Software Requirements

Hardware requirements

- Any desktop or laptop computer with a dual-core processor (Intel, AMD

or ARM).

- At least 2 GB RAM (4 GB recommended).
- A display with a resolution of 1000 x 650 or higher (the window opens at 1200 x 750 and has a minimum size of 1000 x 650).
- About 50 MB of free disk space for Python, the program and the attendance.db file.

Software requirements

- Operating system: Windows, Linux or macOS.
- Python 3.8 or later, including the standard-library modules tkinter, sqlite3, csv and datetime.
- SQLite (bundled with Python); no separate database server is needed.
- A code editor or IDE such as IDLE, VS Code or PyCharm, and Microsoft Excel or any spreadsheet program to open exported CSV files (optional).

4.3 Working Principle

The Employee Attendance Analyzer works by collecting one attendance entry at a time through the Mark Attendance form, validating it, storing it in SQLite and then answering every report with SQL queries over the stored records. After each successful change, refresh_all() reloads the dashboard, the records table and all reports so that every screen always shows current data.

Data model

Table 1: Database tables used by the application

Table	Column	Type	Purpose
employees	employee_id	TEXT, PRIMARY KEY	Unique whole-number ID of the employee
employees	employee_name	TEXT, NOT NULL	Name registered for that ID
attendance	id	INTEGER, PRIMARY KEY AUTOINCREMENT	Unique record number
attendance	employee_id	TEXT	Links the record to an employee
attendance	employee_name	TEXT, NOT NULL	Employee name stored with the record
attendance	attendance_date	TEXT, NOT NULL	Date in YYYY-MM-DD format
attendance	status	TEXT, NOT NULL	Present or Absent
attendance	reason	TEXT	Reason for absence (empty when Present)

Saving a record

When the user presses Save Attendance, save_attendance() applies the rules in Table 2 in order and stops at the first failure with a warning dialog. A new employee ID is registered automatically in the employees table the first time it is saved. If the ID is already registered, the name typed must match the registered name, and if the name box is left empty it is filled in automatically from the database.

Table 2: Validation rules applied before a record is saved

No.	Rule	Message shown
1	Employee ID must not be empty	Please enter a unique employee ID.
2	Employee ID must be a positive whole number	Enter a positive whole-number employee ID.
3	Employee name must not be empty (auto-filled for a registered ID)	Please enter employee name.
4	Date must be a valid YYYY-MM-DD date	Use YYYY-MM-DD format.
5	A status (Present or Absent) must be selected	Please select Present or Absent.
6	A reason is mandatory when the status is Absent (it is cleared when Present)	Please enter the reason for absence.
7	An ID cannot be reused with a different name	ID n is already assigned to <name>.
8	Only one record per employee per date	Attendance ... already exists.

Updating and deleting

Right-clicking a row in the All Attendance Records table opens a context menu with Update Attendance and Delete Attendance. Update loads the row into the form and changes the button text to Update Attendance; on saving, the program checks that the employee ID and name were not changed and that the new date does not duplicate another record, and then runs an UPDATE query. Delete asks for confirmation and then removes the row.

Report calculations

Every report counts records with SUM(CASE WHEN status='Present' THEN 1 ELSE 0 END) and the corresponding expression for Absent, grouped by employee. The attendance percentage is Present / Total x 100. The weekly report converts any date entered into the Monday-to-Sunday range of its week using weekday(), and the monthly report selects rows whose first seven date characters equal the chosen YYYY-MM value. The ranking sorts employees by percentage in descending order (ties broken by name and ID) and reports the highest and lowest values on the dashboard.

Example problem

The Generate Test Data (2 Months) button fills every registered employee with weekday records for the two calendar months before the current month and skips any employee/date

pair that already exists. An employee is marked Absent (reason "Personal leave") when (employee_index + day_of_year) % 17 == 0. For six employees on 1 August to 30 September 2026 (43 weekdays), this creates 258 records, of which 241 are Present and 17 are Absent, giving an overall attendance of 93.4%.

Table 3: Attendance ranking for the sample data (all 258 records)

Employee ID	Employee	Present	Absent	Total	Percentage
4	Priya Nair	41	2	43	95.3%
1	Ravi Kumar	41	2	43	95.3%
2	Anita Sharma	40	3	43	93.0%
5	Imran Khan	40	3	43	93.0%
3	Suresh Reddy	40	3	43	93.0%
6	Kavya Rao	39	4	43	90.7%

4.4 Software Implementation

The complete source code of the application is given below (attendance_analyzer.py). It is a single file organised into labelled sections: colours and main window, database set-up and migration, helper functions, dashboard, add/update/delete attendance, the date, weekly, monthly and ranking reports, navigation, and the sidebar and page layouts.

```
import tkinter as tk
from tkinter import ttk, messagebox, filedialog
import csv
import sqlite3
from datetime import datetime, timedelta

DB_NAME = "attendance.db"

# -------------------- COLORS --------------------
BG = "#f4f6f8"
SIDEBAR = "#172033"
CARD = "#ffffff"
TEXT = "#172033"
MUTED = "#6b7280"
BLUE = "#2563eb"
GREEN = "#16a34a"
RED = "#dc2626"
BORDER = "#e5e7eb"

# -------------------- MAIN WINDOW --------------------
root = tk.Tk()
root.title("Employee Attendance Analyzer")
root.geometry("1200x750")
root.minsize(1000, 650)

# -------------------- DATABASE --------------------
def connect_db():
    return sqlite3.connect(DB_NAME)

def setup_database():
    conn = connect_db()
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS attendance (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            employee_name TEXT NOT NULL,
            attendance_date TEXT NOT NULL,
            status TEXT NOT NULL,
            reason TEXT
        )
    """)
    cur.execute("""
        CREATE TABLE IF NOT EXISTS employees (
            employee_id TEXT PRIMARY KEY,
            employee_name TEXT NOT NULL
        )
    """)

    attendance_columns = {
        row[1] for row in cur.execute("PRAGMA table_info(attendance)")
    }
    if "employee_id" not in attendance_columns:
        cur.execute("ALTER TABLE attendance ADD COLUMN employee_id TEXT")

    employee_ids = {
        row[0] for row in cur.execute("SELECT employee_id FROM employees")
        if row[0] and row[0].isdigit()
    }
    next_employee_number = 1
    nonnumeric_ids = cur.execute(
        "SELECT employee_id FROM employees ORDER BY employee_id"
    ).fetchall()
    for (old_employee_id,) in nonnumeric_ids:
        if old_employee_id and old_employee_id.isdigit():
            continue
        while str(next_employee_number) in employee_ids:
            next_employee_number += 1
        new_employee_id = str(next_employee_number)
```

```
            cur.execute(
                "UPDATE attendance SET employee_id=? WHERE employee_id=?",
                (new_employee_id, old_employee_id)
            )
            cur.execute(
                "UPDATE employees SET employee_id=? WHERE employee_id=?",
                (new_employee_id, old_employee_id)
            )
            employee_ids.add(new_employee_id)
            next_employee_number += 1

        legacy_names = cur.execute(
            """SELECT DISTINCT employee_name FROM attendance
            WHERE employee_id IS NULL OR employee_id='' ORDER BY employee_name"""
        ).fetchall()
        for (name,) in legacy_names:
            employee = cur.execute(
                "SELECT employee_id FROM employees WHERE employee_name=? LIMIT 1",
                (name,)
            ).fetchone()
            if employee:
                employee_id = employee[0]
            else:
                while str(next_employee_number) in employee_ids:
                    next_employee_number += 1
                employee_id = str(next_employee_number)
                cur.execute(
                    "INSERT INTO employees (employee_id, employee_name) VALUES (?, ?)",
                    (employee_id, name)
                )
                employee_ids.add(employee_id)
                next_employee_number += 1
            cur.execute(
                """UPDATE attendance SET employee_id=?
                WHERE employee_name=? AND (employee_id IS NULL OR employee_id='')""",
                (employee_id, name)
            )

        conn.commit()
        conn.close()

    # -------------------- HELPERS --------------------
    def today():
        return datetime.now().strftime("%Y-%m-%d")

    def refresh_employee_ids():
        conn = connect_db()
        cur = conn.cursor()
        cur.execute(
            "SELECT employee_id FROM employees "
            "ORDER BY CAST(employee_id AS INTEGER), employee_id"
        )
        employee_ids = [row[0] for row in cur.fetchall()]
        conn.close()
        employee_id_entry["values"] = employee_ids

    auto_filled_employee_name = None

    def load_employee_name(event=None):
        global auto_filled_employee_name

        employee_id = employee_id_entry.get().strip()
        if not employee_id:
            if (
                auto_filled_employee_name
                and employee_name_entry.get().strip() == auto_filled_employee_name
            ):
                employee_name_entry.delete(0, tk.END)
            auto_filled_employee_name = None
            return

        conn = connect_db()
        cur = conn.cursor()
        cur.execute(
            "SELECT employee_name FROM employees WHERE employee_id=?",
            (employee_id,)
        )
        employee = cur.fetchone()
        conn.close()

```

```
    if employee:
        employee_name_entry.delete(0, tk.END)
        employee_name_entry.insert(0, employee[0])
        auto_filled_employee_name = employee[0]
    elif (
        auto_filled_employee_name
        and employee_name_entry.get().strip() == auto_filled_employee_name
    ):
        employee_name_entry.delete(0, tk.END)
        auto_filled_employee_name = None

def valid_date(value):
    try:
        datetime.strptime(value, "%Y-%m-%d")
        return True
    except ValueError:
        return False

def clear_tree(tree):
    for item in tree.get_children():
        tree.delete(item)

def update_report_empty_state(tree, label, message):
    if tree.get_children():
        label.place_forget()
    else:
        label.config(text=message)
        label.place(relx=0.5, rely=0.5, anchor="center")

def export_report_to_csv(tree, default_filename):
    rows = tree.get_children()
    if not rows:
        messagebox.showwarning("No Report Data", "Generate a report before exporting it.")
        return

    file_path = filedialog.asksaveasfilename(
        title="Export Report to CSV",
        defaultextension=".csv",
        initialfile=default_filename,
        filetypes=[("CSV files", "*.csv"), ("All files", "*.*")]
    )
    if not file_path:
        return

    try:
        with open(file_path, "w", newline="", encoding="utf-8-sig") as csv_file:
            writer = csv.writer(csv_file)
            writer.writerow(tree["columns"])
            for item in rows:
                writer.writerow(tree.item(item, "values"))
    except OSError as error:
        messagebox.showerror("Export Failed", f"Could not save the CSV file.\n{error}")
        return

    messagebox.showinfo("Export Complete", f"Report saved to:\n{file_path}")

def refresh_all():
    refresh_dashboard()
    load_all_records()
    load_date_report()
    load_weekly_report()
    load_monthly_report()
    load_ranking()

# -------------------- DASHBOARD --------------------
def refresh_dashboard():
    conn = connect_db()
    cur = conn.cursor()

    cur.execute("SELECT COUNT(*) FROM employees")
    employees = cur.fetchone()[0]

    cur.execute("SELECT COUNT(*) FROM attendance")
    records = cur.fetchone()[0]

    cur.execute("SELECT COUNT(*) FROM attendance WHERE status='Present'")
    present = cur.fetchone()[0]

    cur.execute("SELECT COUNT(*) FROM attendance WHERE status='Absent'")
    absent = cur.fetchone()[0]
```

```
229
230     conn.close()
231
232     percentage = (present / records * 100) if records else 0
233
234     employee_count_label.config(text=str(employees))
235     total_record_label.config(text=str(records))
236     present_count_label.config(text=str(present))
237     absent_count_label.config(text=str(absent))
238     overall_percentage_label.config(text=f"{percentage:.1f}%")
239
240 # -------------------- ADD / UPDATE ATTENDANCE --------------------
241 editing_record_id = None
242
243 def save_attendance():
244     employee_id = employee_id_entry.get().strip()
245     name = employee_name_entry.get().strip()
246     date = date_entry.get().strip()
247     status = status_combo.get()
248     reason = reason_entry.get().strip()
249
250     if not employee_id:
251         messagebox.showwarning("Missing Information", "Please enter a unique employee ID.")
252         return
253
254     if not employee_id.isdigit() or int(employee_id) < 1:
255         messagebox.showwarning("Invalid Employee ID", "Enter a positive whole-number employee ID.")
256         return
257
258     if not name:
259         conn = connect_db()
260         cur = conn.cursor()
261         cur.execute(
262             "SELECT employee_name FROM employees WHERE employee_id=?",
263             (employee_id,)
264         )
265         registered_employee_name = cur.fetchone()
266         conn.close()
267         if registered_employee_name:
268             name = registered_employee_name[0]
269             employee_name_entry.insert(0, name)
270
271     if not name:
272         messagebox.showwarning("Missing Information", "Please enter employee name.")
273         return
274
275     if not valid_date(date):
276         messagebox.showwarning(
277             "Invalid Date",
278             "Use YYYY-MM-DD format.\nExample: 2026-10-03"
279         )
280         return
281
282     if not status:
283         messagebox.showwarning("Missing Information", "Please select Present or Absent.")
284         return
285
286     if status == "Absent" and not reason:
287         messagebox.showwarning(
288             "Missing Reason",
289             "Please enter the reason for absence."
290         )
291         return
292
293     if status == "Present":
294         reason = ""
295
296     conn = connect_db()
297     cur = conn.cursor()
298
299     cur.execute(
300         "SELECT employee_name FROM employees WHERE employee_id=?",
301         (employee_id,)
302     )
303     registered_employee = cur.fetchone()
304     if registered_employee and registered_employee[0] != name:
305         conn.close()
306         messagebox.showwarning(
307             "Employee ID In Use",
308             f"ID {employee_id} is already assigned to {registered_employee[0]}."
```

```
309             )
310             return
311 
312         if editing_record_id is not None:
313             cur.execute(
314                 "SELECT employee_id, employee_name FROM attendance WHERE id=?",
315                 (editing_record_id,)
316             )
317             loaded_record = cur.fetchone()
318             if not loaded_record or loaded_record != (employee_id, name):
319                 conn.close()
320                 messagebox.showwarning(
321                     "Employee Changed",
322                     "Clear the form and load the correct employee record before updating."
323                 )
324                 return
325 
326             cur.execute(
327                 """SELECT id FROM attendance
328                 WHERE employee_id=? AND attendance_date=? AND id<>?""",
329                 (employee_id, date, editing_record_id)
330             )
331             duplicate = cur.fetchone()
332             if duplicate:
333                 conn.close()
334                 messagebox.showwarning(
335                     "Record Exists",
336                     "This employee already has an attendance record for that date."
337                 )
338                 return
339 
340             conn.close()
341             update_record(editing_record_id, date, status, reason)
342             return
343 
344         cur.execute("""
345             SELECT id FROM attendance
346             WHERE employee_id=? AND attendance_date=?
347         """, (employee_id, date))
348 
349         existing = cur.fetchone()
350 
351         if existing:
352             conn.close()
353             messagebox.showwarning(
354                 "Record Exists",
355                 f"Attendance for {name} (ID {employee_id}) on {date} already exists. "
356                 "Right-click its row and choose Update Attendance to edit it."
357             )
358             return
359 
360         if not registered_employee:
361             cur.execute(
362                 "INSERT INTO employees (employee_id, employee_name) VALUES (?, ?)",
363                 (employee_id, name)
364             )
365 
366         cur.execute("""
367             INSERT INTO attendance
368             (employee_id, employee_name, attendance_date, status, reason)
369             VALUES (?, ?, ?, ?, ?)
370         """, (employee_id, name, date, status, reason))
371 
372         conn.commit()
373         conn.close()
374 
375         refresh_employee_ids()
376         clear_form()
377         refresh_all()
378         messagebox.showinfo("Saved", "Attendance saved successfully.")
379 
380     def generate_sample_attendance():
381         conn = connect_db()
382         cur = conn.cursor()
383         cur.execute(
384             "SELECT employee_id, employee_name FROM employees "
385             "ORDER BY CAST(employee_id AS INTEGER), employee_id"
386         )
387         employees = cur.fetchall()
388 
```

```
    if not employees:
        conn.close()
        messagebox.showwarning(
            "No Employees",
            "Add employees before generating sample attendance."
        )
        return

    current_month_start = datetime.now().replace(day=1).date()
    end_date = current_month_start - timedelta(days=1)
    previous_month_start = end_date.replace(day=1)
    start_date = (previous_month_start - timedelta(days=1)).replace(day=1)

    weekdays = []
    attendance_date = start_date
    while attendance_date <= end_date:
        if attendance_date.weekday() < 5:
            weekdays.append(attendance_date)
        attendance_date += timedelta(days=1)

    if not messagebox.askyesno(
        "Generate Test Attendance",
        "Generate sample attendance for all registered employees on weekdays "
        f"from {start_date:%Y-%m-%d} to {end_date:%Y-%m-%d}?\\n\\n"
        "Existing employee/date records will be kept unchanged."
    ):
        conn.close()
        return

    cur.execute(
        """SELECT employee_id, attendance_date FROM attendance
        WHERE attendance_date BETWEEN ? AND ?""",
        (start_date.strftime("%Y-%m-%d"), end_date.strftime("%Y-%m-%d"))
    )
    existing_records = set(cur.fetchall())
    new_records = []

    for employee_index, (employee_id, name) in enumerate(employees):
        for attendance_date in weekdays:
            date_text = attendance_date.strftime("%Y-%m-%d")
            if (employee_id, date_text) in existing_records:
                continue

            is_absent = (employee_index + attendance_date.timetuple().tm_yday) % 17 == 0
            status = "Absent" if is_absent else "Present"
            reason = "Personal leave" if is_absent else ""
            new_records.append(
                (employee_id, name, date_text, status, reason)
            )

    cur.executemany(
        """INSERT INTO attendance
        (employee_id, employee_name, attendance_date, status, reason)
        VALUES (?, ?, ?, ?, ?)""",
        new_records
    )
    conn.commit()
    conn.close()

    refresh_all()
    messagebox.showinfo(
        "Test Attendance Generated",
        f"Added {len(new_records)} attendance records for "
        f"{len(employees)} employees from {start_date:%Y-%m-%d} "
        f"to {end_date:%Y-%m-%d}. Existing records were not changed."
    )

def update_record(record_id, date, status, reason):
    conn = connect_db()
    cur = conn.cursor()

    cur.execute("""
        UPDATE attendance
        SET attendance_date=?, status=?, reason=?
        WHERE id=?
    """, (date, status, reason, record_id))

    conn.commit()
    conn.close()
    clear_form()
```

```
    refresh_all()
    messagebox.showinfo("Updated", "Attendance updated successfully.")

def clear_form():
    global editing_record_id
    global auto_filled_employee_name

    employee_id_entry.delete(0, tk.END)
    employee_name_entry.delete(0, tk.END)
    auto_filled_employee_name = None
    date_entry.delete(0, tk.END)
    date_entry.insert(0, today())
    status_combo.set("")
    reason_entry.delete(0, tk.END)
    editing_record_id = None
    save_attendance_button.config(text="Save Attendance")

# ------------------- ALL RECORDS -------------------
def load_all_records():
    clear_tree(all_records_tree)

    conn = connect_db()
    cur = conn.cursor()
    cur.execute("""
        SELECT id, employee_id, employee_name, attendance_date, status, COALESCE(reason,'')
        FROM attendance
        ORDER BY attendance_date DESC, CAST(employee_id AS INTEGER), employee_id
    """)
    rows = cur.fetchall()
    conn.close()

    for record_id, *values in rows:
        all_records_tree.insert("", tk.END, iid=str(record_id), values=values)

def delete_selected_attendance():
    selected = all_records_tree.selection()
    if not selected:
        return

    record_id = selected[0]
    employee_id, name, date = all_records_tree.item(record_id, "values")[:3]
    if not messagebox.askyesno(
        "Delete Attendance",
        f"Delete {name} (ID {employee_id}) attendance for {date}?"
    ):
        return

    conn = connect_db()
    cur = conn.cursor()
    cur.execute("DELETE FROM attendance WHERE id=?", (record_id,))
    conn.commit()
    conn.close()

    refresh_all()
    messagebox.showinfo("Deleted", "Attendance record deleted.")

def update_selected_attendance():
    global editing_record_id

    selected = all_records_tree.selection()
    if not selected:
        return

    record_id = selected[0]
    employee_id, name, date, status, reason = all_records_tree.item(
        record_id, "values"
    )
    editing_record_id = int(record_id)

    employee_id_entry.delete(0, tk.END)
    employee_id_entry.insert(0, employee_id)
    employee_name_entry.delete(0, tk.END)
    employee_name_entry.insert(0, name)
    date_entry.delete(0, tk.END)
    date_entry.insert(0, date)
    status_combo.set(status)
    reason_entry.delete(0, tk.END)
    reason_entry.insert(0, reason)
    save_attendance_button.config(text="Update Attendance")

```

```
def show_attendance_context_menu(event):
    row_id = all_records_tree.identify_row(event.y)
    if not row_id:
        return

    all_records_tree.selection_set(row_id)
    try:
        attendance_context_menu.tk_popup(event.x_root, event.y_root)
    finally:
        attendance_context_menu.grab_release()

# -------------------- DATE REPORT --------------------
def load_date_report():
    date = search_date_entry.get().strip()

    if not valid_date(date):
        date = today()
        search_date_entry.delete(0, tk.END)
        search_date_entry.insert(0, date)

    clear_tree(date_report_tree)

    conn = connect_db()
    cur = conn.cursor()

    cur.execute("""
        SELECT employee_id, employee_name, attendance_date, status, COALESCE(reason,'')
        FROM attendance
        WHERE attendance_date=?
        ORDER BY CAST(employee_id AS INTEGER), employee_id
    """, (date,))
    rows = cur.fetchall()

    cur.execute("""
        SELECT COUNT(*) FROM attendance
        WHERE attendance_date=? AND status='Present'
    """, (date,))
    present = cur.fetchone()[0]

    cur.execute("""
        SELECT COUNT(*) FROM attendance
        WHERE attendance_date=? AND status='Absent'
    """, (date,))
    absent = cur.fetchone()[0]

    conn.close()

    date_report_title.config(
        text=f"Date: {date}    |    Present: {present}    |    Absent: {absent}"
    )

    for row in rows:
        date_report_tree.insert("", tk.END, values=row)
    update_report_empty_state(
        date_report_tree,
        date_report_empty_label,
        f"No attendance records found for {date}."
    )

# -------------------- WEEKLY REPORT --------------------
def load_weekly_report():
    value = week_entry.get().strip()

    if not valid_date(value):
        messagebox.showwarning(
            "Invalid Date",
            "Enter any date in the week using YYYY-MM-DD."
        )
        return

    selected = datetime.strptime(value, "%Y-%m-%d")
    monday = selected - timedelta(days=selected.weekday())
    sunday = monday + timedelta(days=6)

    start = monday.strftime("%Y-%m-%d")
    end = sunday.strftime("%Y-%m-%d")

    clear_tree(weekly_tree)

    conn = connect_db()
```

```
    cur = conn.cursor()

    cur.execute("""
        SELECT employee_id, employee_name,
               SUM(CASE WHEN status='Present' THEN 1 ELSE 0 END),
               SUM(CASE WHEN status='Absent' THEN 1 ELSE 0 END),
               COUNT(*)
        FROM attendance
        WHERE attendance_date BETWEEN ? AND ?
        GROUP BY employee_id, employee_name
        ORDER BY CAST(employee_id AS INTEGER), employee_id
    """, (start, end))

    rows = cur.fetchall()
    conn.close()

    weekly_title.config(
        text=f"Weekly Attendance — {start} to {end}"
    )

    for employee_id, name, present, absent, total in rows:
        percentage = present / total * 100 if total else 0
        weekly_tree.insert(
            "", tk.END,
            values=(employee_id, name, present, absent, total, f"{percentage:.1f}%")
        )
    update_report_empty_state(
        weekly_tree,
        weekly_empty_label,
        f"No attendance records found from {start} to {end}."
    )

# -------------------- MONTHLY REPORT --------------------
def load_monthly_report():
    value = month_entry.get().strip()

    try:
        month = datetime.strptime(value, "%Y-%m").strftime("%Y-%m")
    except ValueError:
        messagebox.showwarning(
            "Invalid Month",
            "Enter month as YYYY-MM.\nExample: 2026-10"
        )
        return

    clear_tree(monthly_tree)

    conn = connect_db()
    cur = conn.cursor()

    cur.execute("""
        SELECT employee_id, employee_name,
               SUM(CASE WHEN status='Present' THEN 1 ELSE 0 END),
               SUM(CASE WHEN status='Absent' THEN 1 ELSE 0 END),
               COUNT(*)
        FROM attendance
        WHERE substr(attendance_date,1,7)=?
        GROUP BY employee_id, employee_name
        ORDER BY CAST(employee_id AS INTEGER), employee_id
    """, (month,))

    rows = cur.fetchall()
    conn.close()

    monthly_title.config(text=f"Monthly Attendance — {month}")

    for employee_id, name, present, absent, total in rows:
        percentage = present / total * 100 if total else 0
        monthly_tree.insert(
            "", tk.END,
            values=(employee_id, name, present, absent, total, f"{percentage:.1f}%")
        )
    update_report_empty_state(
        monthly_tree,
        monthly_empty_label,
        f"No attendance records found for {month}."
    )

# -------------------- RANKING --------------------
def load_ranking():
```

```
    clear_tree(ranking_tree)

    conn = connect_db()
    cur = conn.cursor()

    cur.execute("""
        SELECT employee_id, employee_name,
               SUM(CASE WHEN status='Present' THEN 1 ELSE 0 END),
               SUM(CASE WHEN status='Absent' THEN 1 ELSE 0 END),
               COUNT(*)
        FROM attendance
        GROUP BY employee_id, employee_name
    """)

    rows = cur.fetchall()
    conn.close()

    data = []

    for employee_id, name, present, absent, total in rows:
        percentage = present / total * 100 if total else 0
        data.append((employee_id, name, present, absent, total, percentage))

    data.sort(key=lambda x: (-x[5], x[1], x[0]))

    for row in data:
        ranking_tree.insert(
            "", tk.END,
            values=(row[0], row[1], row[2], row[3], row[4], f"{row[5]:.1f}%")
        )

    if data:
        highest = max(data, key=lambda x: x[5])
        lowest = min(data, key=lambda x: x[5])

        highest_label.config(
            text=f"Highest Attendance\n{highest[1]} (ID {highest[0]}) — {highest[5]:.1f}%"
        )
        lowest_label.config(
            text=f"Lowest Attendance\n{lowest[1]} (ID {lowest[0]}) — {lowest[5]:.1f}%"
        )
    else:
        highest_label.config(text="Highest Attendance\nNo data")
        lowest_label.config(text="Lowest Attendance\nNo data")

# -------------------- NAVIGATION --------------------
pages = {}

def show_page(name):
    for page in pages.values():
        page.pack_forget()

    pages[name].pack(fill="both", expand=True)

    if name == "Dashboard":
        refresh_dashboard()
        load_ranking()
    elif name == "Attendance":
        load_all_records()
    elif name == "Date Report":
        load_date_report()
    elif name == "Weekly Report":
        load_weekly_report()
    elif name == "Monthly Report":
        load_monthly_report()

# ============================================================
# SIDEBAR
# ============================================================
sidebar = tk.Frame(root, bg=SIDEBAR, width=230)
sidebar.pack(side="left", fill="y")
sidebar.pack_propagate(False)

tk.Label(
    sidebar,
    text="Attendance\nAnalyzer",
    bg=SIDEBAR,
    fg=CARD,
    font=("Arial", 23, "bold"),
    justify="left"
```

```
).pack(padx=25, pady=(35, 25), anchor="w")

tk.Label(
    sidebar,
    text="REPORTS & TOOLS",
    bg=SIDEBAR,
    fg="#9ca3af",
    font=("Arial", 9, "bold")
).pack(padx=25, pady=(0, 12), anchor="w")

def nav_button(text, page):
    tk.Button(
        sidebar,
        text=text,
        command=lambda: show_page(page),
        bg=SIDEBAR,
        fg="#d1d5db",
        activebackground="#27344d",
        activeforeground=CARD,
        font=("Arial", 11),
        relief="flat",
        bd=0,
        anchor="w",
        padx=25,
        pady=11,
        cursor="hand2"
    ).pack(fill="x")

nav_button("  Dashboard", "Dashboard")
nav_button("  Mark Attendance", "Attendance")
nav_button("  Date-wise Search", "Date Report")
nav_button("  Weekly Report", "Weekly Report")
nav_button("  Monthly Report", "Monthly Report")

# ============================================================
# CONTENT AREA
# ============================================================
content = tk.Frame(root, bg=BG)
content.pack(side="left", fill="both", expand=True)

# ============================================================
# DASHBOARD
# ============================================================
dashboard_page = tk.Frame(content, bg=BG)
pages["Dashboard"] = dashboard_page

tk.Label(
    dashboard_page,
    text="Attendance Dashboard",
    bg=BG, fg=TEXT,
    font=("Arial", 25, "bold")
).pack(anchor="w", padx=30, pady=(25, 3))

tk.Label(
    dashboard_page,
    text="Overview of employee attendance and performance",
    bg=BG, fg=MUTED,
    font=("Arial", 11)
).pack(anchor="w", padx=30)

cards = tk.Frame(dashboard_page, bg=BG)
cards.pack(fill="x", padx=25, pady=18)

def stat_card(parent, title, value):
    frame = tk.Frame(
        parent, bg=CARD,
        highlightthickness=1,
        highlightbackground=BORDER
    )
    frame.pack(side="left", expand=True, fill="both", padx=6)

    tk.Label(
        frame, text=title,
        bg=CARD, fg=MUTED,
        font=("Arial", 10, "bold")
    ).pack(anchor="w", padx=18, pady=(18, 5))

    label = tk.Label(
        frame, text=value,
        bg=CARD, fg=TEXT,
```

```
            font=("Arial", 23, "bold")
        )
        label.pack(anchor="w", padx=18, pady=(0, 18))
        return label

    employee_count_label = stat_card(cards, "EMPLOYEES", "0")
    total_record_label = stat_card(cards, "TOTAL RECORDS", "0")
    present_count_label = stat_card(cards, "PRESENT", "0")
    absent_count_label = stat_card(cards, "ABSENT", "0")
    overall_percentage_label = stat_card(cards, "OVERALL ATTENDANCE", "0%")

    ranking_frame = tk.Frame(
        dashboard_page, bg=CARD,
        highlightthickness=1,
        highlightbackground=BORDER
    )
    ranking_frame.pack(fill="both", expand=True, padx=30, pady=10)

    tk.Label(
        ranking_frame,
        text="Employee Attendance Ranking",
        bg=CARD, fg=TEXT,
        font=("Arial", 16, "bold")
    ).pack(anchor="w", padx=20, pady=(18, 3))

    tk.Label(
        ranking_frame,
        text="Highest and lowest attendance are calculated from all saved records.",
        bg=CARD, fg=MUTED,
        font=("Arial", 9)
    ).pack(anchor="w", padx=20, pady=(0, 10))

    ranking_tree = ttk.Treeview(
        ranking_frame,
        columns=("Employee ID", "Employee", "Present", "Absent", "Total", "Percentage"),
        show="headings"
    )

    for col, width in [
        ("Employee ID", 130), ("Employee", 200), ("Present", 100),
        ("Absent", 100), ("Total", 100), ("Percentage", 130)
    ]:
        ranking_tree.heading(col, text=col)
        ranking_tree.column(col, width=width)

    ranking_tree.pack(fill="both", expand=True, padx=20, pady=(0, 10))

    summary = tk.Frame(ranking_frame, bg=CARD)
    summary.pack(fill="x", padx=20, pady=(0, 18))

    highest_label = tk.Label(
        summary,
        text="Highest Attendance\nNo data",
        bg="#ecfdf5", fg=GREEN,
        font=("Arial", 11, "bold"),
        pady=12
    )
    highest_label.pack(side="left", fill="x", expand=True, padx=(0, 5))

    lowest_label = tk.Label(
        summary,
        text="Lowest Attendance\nNo data",
        bg="#fef2f2", fg=RED,
        font=("Arial", 11, "bold"),
        pady=12
    )
    lowest_label.pack(side="left", fill="x", expand=True, padx=(5, 0))

    # ============================================================
    # ATTENDANCE PAGE
    # ============================================================
    attendance_page = tk.Frame(content, bg=BG)
    pages["Attendance"] = attendance_page

    tk.Label(
        attendance_page,
        text="Mark Attendance",
        bg=BG, fg=TEXT,
        font=("Arial", 25, "bold")
    ).pack(anchor="w", padx=30, pady=(25, 3))
```

```
949
950  tk.Label(
951      attendance_page,
952      text="Record present/absent status and the reason for absence.",
953      bg=BG, fg=MUTED,
954      font=("Arial", 11)
955  ).pack(anchor="w", padx=30)
956
957  form = tk.Frame(
958      attendance_page, bg=CARD,
959      highlightthickness=1,
960      highlightbackground=BORDER
961  )
962  form.pack(fill="x", padx=30, pady=15)
963
964  tk.Label(form, text="Employee ID", bg=CARD, fg=TEXT,
965           font=("Arial", 10, "bold")).grid(row=0, column=0, padx=15, pady=(18, 6), sticky="w")
966  employee_id_entry = ttk.Combobox(form, width=14)
967  employee_id_entry.grid(row=1, column=0, padx=15, pady=(0, 18))
968  employee_id_entry.bind("<<ComboboxSelected>>", load_employee_name)
969  employee_id_entry.bind("<FocusOut>", load_employee_name)
970  employee_id_entry.bind("<KeyRelease>", load_employee_name)
971
972  tk.Label(form, text="Employee Name", bg=CARD, fg=TEXT,
973           font=("Arial", 10, "bold")).grid(row=0, column=1, padx=15, pady=(18, 6), sticky="w")
974  employee_name_entry = ttk.Entry(form, width=20)
975  employee_name_entry.grid(row=1, column=1, padx=15, pady=(0, 18))
976
977  tk.Label(form, text="Date (YYYY-MM-DD)", bg=CARD, fg=TEXT,
978           font=("Arial", 10, "bold")).grid(row=0, column=2, padx=15, pady=(18, 6), sticky="w")
979  date_entry = ttk.Entry(form, width=16)
980  date_entry.grid(row=1, column=2, padx=15, pady=(0, 18))
981  date_entry.insert(0, today())
982
983  tk.Label(form, text="Status", bg=CARD, fg=TEXT,
984           font=("Arial", 10, "bold")).grid(row=0, column=3, padx=15, pady=(18, 6), sticky="w")
985  status_combo = ttk.Combobox(
986      form, values=["Present", "Absent"],
987      state="readonly", width=12
988  )
989  status_combo.grid(row=1, column=3, padx=15, pady=(0, 18))
990
991  tk.Label(form, text="Absence Reason", bg=CARD, fg=TEXT,
992           font=("Arial", 10, "bold")).grid(row=0, column=4, padx=15, pady=(18, 6), sticky="w")
993  reason_entry = ttk.Entry(form, width=20)
994  reason_entry.grid(row=1, column=4, padx=15, pady=(0, 18))
995
996  save_attendance_button = tk.Button(
997      form,
998      text="Save Attendance",
999      command=save_attendance,
1000     bg=BLUE, fg=CARD,
1001     activebackground="#1d4ed8",
1002     font=("Arial", 10, "bold"),
1003     relief="flat", padx=16, pady=9,
1004     cursor="hand2"
1005 )
1006 save_attendance_button.grid(row=2, column=5, padx=15, pady=(0, 18))
1007
1008 tk.Button(
1009     form,
1010     text="Generate Test Data (2 Months)",
1011     command=generate_sample_attendance,
1012     bg=GREEN, fg=CARD,
1013     activebackground="#15803d",
1014     font=("Arial", 10, "bold"),
1015     relief="flat", padx=16, pady=9,
1016     cursor="hand2"
1017 ).grid(row=2, column=4, padx=15, pady=(0, 18), sticky="w")
1018
1019 records_frame = tk.Frame(
1020     attendance_page, bg=CARD,
1021     highlightthickness=1,
1022     highlightbackground=BORDER
1023 )
1024 records_frame.pack(fill="both", expand=True, padx=30, pady=(0, 25))
1025
1026 tk.Label(
1027     records_frame,
1028     text="All Attendance Records",
```

```
    bg=CARD, fg=TEXT,
    font=("Arial", 16, "bold")
).pack(anchor="w", padx=20, pady=15)

all_records_tree = ttk.Treeview(
    records_frame,
    columns=("Employee ID", "Employee", "Date", "Status", "Reason"),
    show="headings"
)

for col, width in [
    ("Employee ID", 130), ("Employee", 200), ("Date", 150),
    ("Status", 120), ("Reason", 350)
]:
    all_records_tree.heading(col, text=col)
    all_records_tree.column(col, width=width)

all_records_tree.pack(fill="both", expand=True, padx=20, pady=(0, 20))

attendance_context_menu = tk.Menu(root, tearoff=0)
attendance_context_menu.add_command(
    label="Update Attendance",
    command=update_selected_attendance
)
attendance_context_menu.add_command(
    label="Delete Attendance",
    command=delete_selected_attendance
)
all_records_tree.bind("<Button-3>", show_attendance_context_menu)

# ============================================================
# DATE REPORT
# ============================================================
date_page = tk.Frame(content, bg=BG)
pages["Date Report"] = date_page

tk.Label(
    date_page,
    text="Date-wise Attendance Search",
    bg=BG, fg=TEXT,
    font=("Arial", 25, "bold")
).pack(anchor="w", padx=30, pady=(25, 3))

tk.Label(
    date_page,
    text="Search a date to find who was absent, who was present, and the absence reasons.",
    bg=BG, fg=MUTED,
    font=("Arial", 11)
).pack(anchor="w", padx=30)

search_bar = tk.Frame(date_page, bg=CARD)
search_bar.pack(fill="x", padx=30, pady=15)

tk.Label(
    search_bar,
    text="Date (YYYY-MM-DD)",
    bg=CARD, fg=TEXT,
    font=("Arial", 10, "bold")
).pack(side="left", padx=(20, 10), pady=18)

search_date_entry = ttk.Entry(search_bar, width=20)
search_date_entry.pack(side="left", pady=18)
search_date_entry.insert(0, today())

tk.Button(
    search_bar,
    text="Search Date",
    command=load_date_report,
    bg=BLUE, fg=CARD,
    font=("Arial", 10, "bold"),
    relief="flat", padx=18, pady=8,
    cursor="hand2"
).pack(side="left", padx=12)

tk.Button(
    search_bar,
    text="Export CSV",
    command=lambda: export_report_to_csv(date_report_tree, "date_attendance_report.csv"),
    bg=GREEN, fg=CARD,
    font=("Arial", 10, "bold"),
```

```
    relief="flat", padx=18, pady=8,
    cursor="hand2"
).pack(side="left", padx=4)

date_report_title = tk.Label(
    date_page,
    text="Date Report",
    bg=BG, fg=TEXT,
    font=("Arial", 15, "bold")
)
date_report_title.pack(anchor="w", padx=30, pady=(5, 8))

date_frame = tk.Frame(
    date_page, bg=CARD,
    highlightthickness=1,
    highlightbackground=BORDER
)
date_frame.pack(fill="both", expand=True, padx=30, pady=(0, 25))

date_report_tree = ttk.Treeview(
    date_frame,
    columns=("Employee ID", "Employee", "Date", "Status", "Reason"),
    show="headings"
)

for col, width in [
    ("Employee ID", 130), ("Employee", 210), ("Date", 150),
    ("Status", 130), ("Reason", 350)
]:
    date_report_tree.heading(col, text=col)
    date_report_tree.column(col, width=width)

date_report_tree.pack(fill="both", expand=True, padx=20, pady=20)
date_report_empty_label = tk.Label(
    date_frame, text="", bg=CARD, fg=MUTED, font=("Arial", 11)
)

# ============================================================
# WEEKLY REPORT
# ============================================================
weekly_page = tk.Frame(content, bg=BG)
pages["Weekly Report"] = weekly_page

tk.Label(
    weekly_page,
    text="Weekly Attendance Report",
    bg=BG, fg=TEXT,
    font=("Arial", 25, "bold")
).pack(anchor="w", padx=30, pady=(25, 3))

tk.Label(
    weekly_page,
    text="View present days, absent days, total records and attendance percentage.",
    bg=BG, fg=MUTED,
    font=("Arial", 11)
).pack(anchor="w", padx=30)

week_bar = tk.Frame(weekly_page, bg=CARD)
week_bar.pack(fill="x", padx=30, pady=15)

tk.Label(
    week_bar,
    text="Any date in the week",
    bg=CARD, fg=TEXT,
    font=("Arial", 10, "bold")
).pack(side="left", padx=(20, 10), pady=18)

week_entry = ttk.Entry(week_bar, width=20)
week_entry.pack(side="left", pady=18)
week_entry.insert(0, today())

tk.Button(
    week_bar,
    text="Generate Weekly Report",
    command=load_weekly_report,
    bg=BLUE, fg=CARD,
    font=("Arial", 10, "bold"),
    relief="flat", padx=18, pady=8,
    cursor="hand2"
).pack(side="left", padx=12)
```

```
1189
1190  tk.Button(
1191      week_bar,
1192      text="Export CSV",
1193      command=lambda: export_report_to_csv(weekly_tree, "weekly_attendance_report.csv"),
1194      bg=GREEN, fg=CARD,
1195      font=("Arial", 10, "bold"),
1196      relief="flat", padx=18, pady=8,
1197      cursor="hand2"
1198  ).pack(side="left", padx=4)
1199
1200  weekly_title = tk.Label(
1201      weekly_page,
1202      text="Weekly Attendance",
1203      bg=BG, fg=TEXT,
1204      font=("Arial", 15, "bold")
1205  )
1206  weekly_title.pack(anchor="w", padx=30, pady=(5, 8))
1207
1208  weekly_frame = tk.Frame(
1209      weekly_page, bg=CARD,
1210      highlightthickness=1,
1211      highlightbackground=BORDER
1212  )
1213  weekly_frame.pack(fill="both", expand=True, padx=30, pady=(0, 25))
1214
1215  weekly_tree = ttk.Treeview(
1216      weekly_frame,
1217      columns=("Employee ID", "Employee", "Present", "Absent", "Total", "Percentage"),
1218      show="headings"
1219  )
1220
1221  for col, width in [
1222      ("Employee ID", 130), ("Employee", 210), ("Present", 100),
1223      ("Absent", 100), ("Total", 100), ("Percentage", 130)
1224  ]:
1225      weekly_tree.heading(col, text=col)
1226      weekly_tree.column(col, width=width)
1227
1228  weekly_tree.pack(fill="both", expand=True, padx=20, pady=20)
1229  weekly_empty_label = tk.Label(
1230      weekly_frame, text="", bg=CARD, fg=MUTED, font=("Arial", 11)
1231  )
1232
1233  # ============================================================
1234  # MONTHLY REPORT
1235  # ============================================================
1236  monthly_page = tk.Frame(content, bg=BG)
1237  pages["Monthly Report"] = monthly_page
1238
1239  tk.Label(
1240      monthly_page,
1241      text="Monthly Attendance Report",
1242      bg=BG, fg=TEXT,
1243      font=("Arial", 25, "bold")
1244  ).pack(anchor="w", padx=30, pady=(25, 3))
1245
1246  tk.Label(
1247      monthly_page,
1248      text="Analyze employee attendance for an entire month.",
1249      bg=BG, fg=MUTED,
1250      font=("Arial", 11)
1251  ).pack(anchor="w", padx=30)
1252
1253  month_bar = tk.Frame(monthly_page, bg=CARD)
1254  month_bar.pack(fill="x", padx=30, pady=15)
1255
1256  tk.Label(
1257      month_bar,
1258      text="Month (YYYY-MM)",
1259      bg=CARD, fg=TEXT,
1260      font=("Arial", 10, "bold")
1261  ).pack(side="left", padx=(20, 10), pady=18)
1262
1263  month_entry = ttk.Entry(month_bar, width=20)
1264  month_entry.pack(side="left", pady=18)
1265  month_entry.insert(0, datetime.now().strftime("%Y-%m"))
1266
1267  tk.Button(
1268      month_bar,
```

```
    text="Generate Monthly Report",
    command=load_monthly_report,
    bg=BLUE, fg=CARD,
    font=("Arial", 10, "bold"),
    relief="flat", padx=18, pady=8,
    cursor="hand2"
).pack(side="left", padx=12)

tk.Button(
    month_bar,
    text="Export CSV",
    command=lambda: export_report_to_csv(monthly_tree, "monthly_attendance_report.csv"),
    bg=GREEN, fg=CARD,
    font=("Arial", 10, "bold"),
    relief="flat", padx=18, pady=8,
    cursor="hand2"
).pack(side="left", padx=4)

monthly_title = tk.Label(
    monthly_page,
    text="Monthly Attendance",
    bg=BG, fg=TEXT,
    font=("Arial", 15, "bold")
)
monthly_title.pack(anchor="w", padx=30, pady=(5, 8))

monthly_frame = tk.Frame(
    monthly_page, bg=CARD,
    highlightthickness=1,
    highlightbackground=BORDER
)
monthly_frame.pack(fill="both", expand=True, padx=30, pady=(0, 25))

monthly_tree = ttk.Treeview(
    monthly_frame,
    columns=("Employee ID", "Employee", "Present", "Absent", "Total", "Percentage"),
    show="headings"
)

for col, width in [
    ("Employee ID", 130), ("Employee", 210), ("Present", 100),
    ("Absent", 100), ("Total", 100), ("Percentage", 130)
]:
    monthly_tree.heading(col, text=col)
    monthly_tree.column(col, width=width)

monthly_tree.pack(fill="both", expand=True, padx=20, pady=20)
monthly_empty_label = tk.Label(
    monthly_frame, text="", bg=CARD, fg=MUTED, font=("Arial", 11)
)

# -------------------- START --------------------
setup_database()
refresh_employee_ids()

refresh_dashboard()
load_all_records()
load_date_report()
load_weekly_report()
load_monthly_report()
load_ranking()

show_page("Dashboard")
root.mainloop()
```

5 Results and Discussion

The figures below show the five screens of the application populated with the sample data produced by the built-in test-data generator (6 employees, 1 August to 30 September 2026, 43 weekdays, 258 records). All numbers shown were produced by the same SQL queries used in the program.

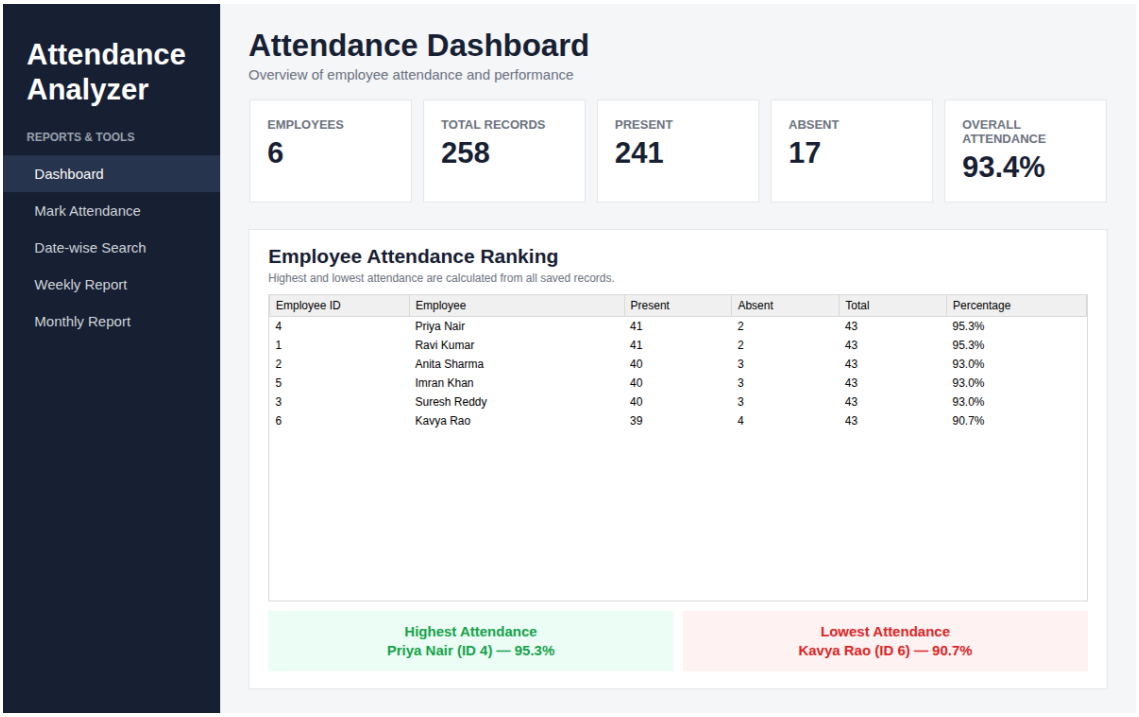


Figure 2: Dashboard with summary cards and attendance ranking

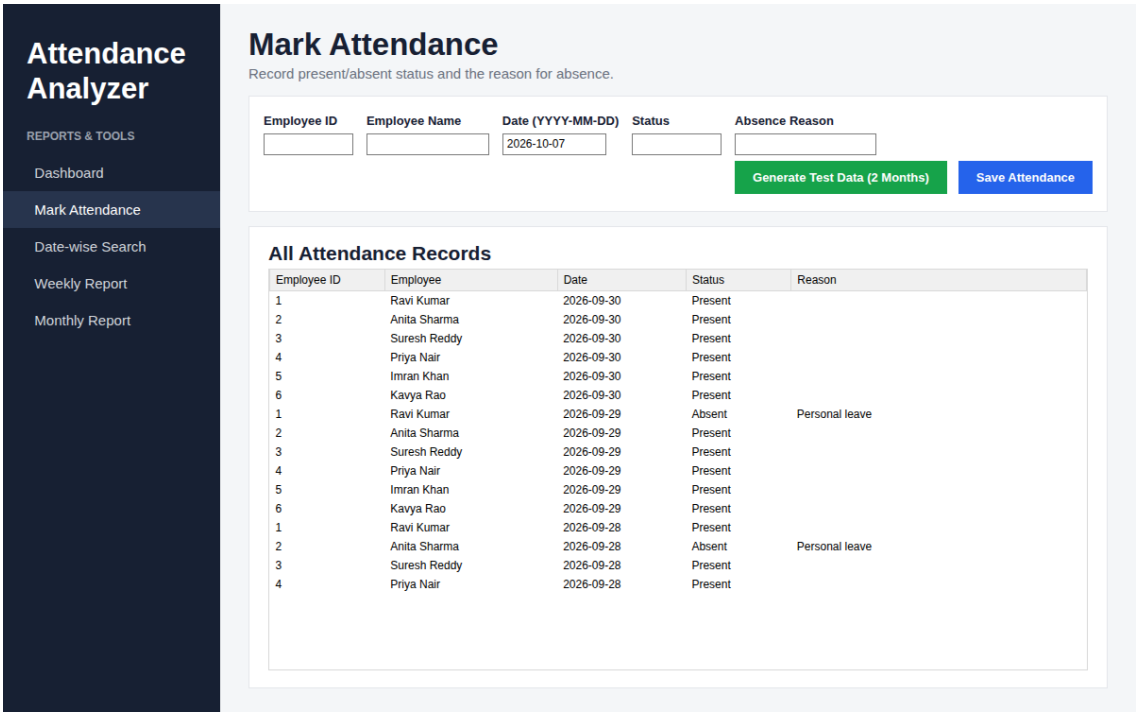


Figure 3: Mark Attendance form and All Attendance Records table

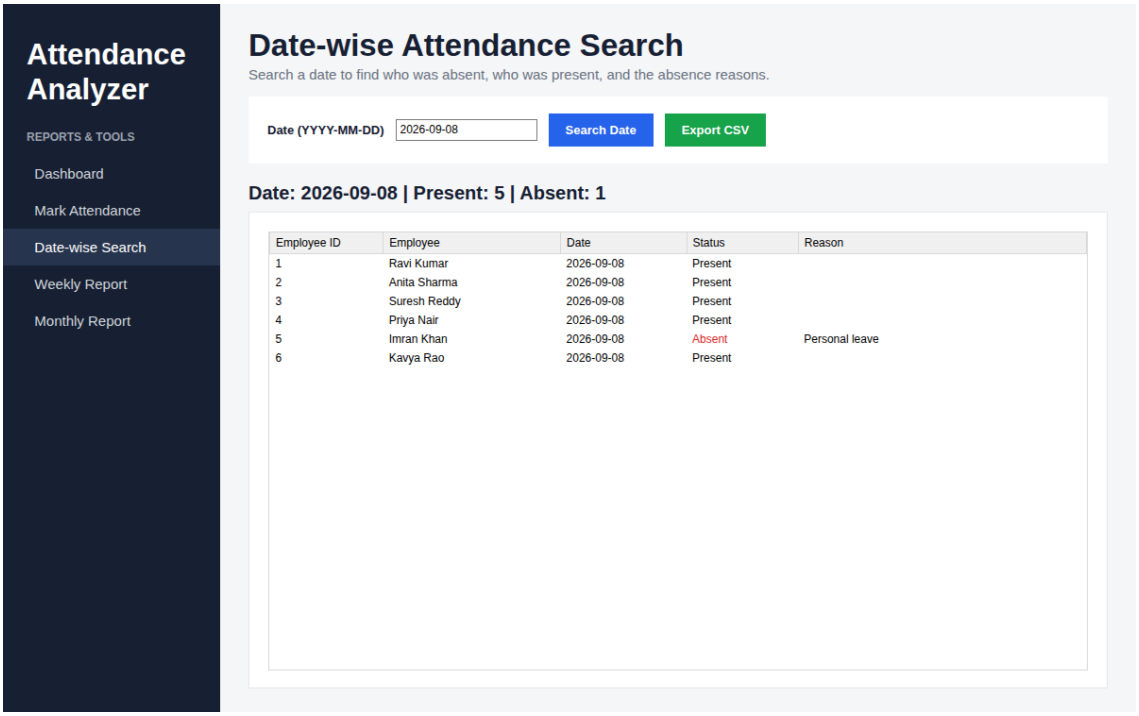


Figure 4: Date-wise search for 2026-09-08 (5 present, 1 absent with reason)



Figure 5: Weekly report for 2026-09-07 to 2026-09-13



Figure 6: Monthly report for 2026-09

Table 4: Functional scenarios and the behaviour implemented

No.	Scenario	Action	Behaviour implemented
1	Empty ID	Press Save with no employee ID	Warning: Please enter a unique employee ID.
2	Invalid ID	Enter abc or 0 as ID	Warning: Enter a positive whole-number employee ID.
3	Registered ID, name blank	Select an existing ID and press Save	Name is filled in from the employees table
4	Invalid date	Enter 07-10-2026	Warning showing the YYYY-MM-DD format with an example
5	Absent without reason	Status Absent, reason empty	Warning: Please enter the reason for absence.
6	ID reused	Existing ID with a different name	Warning: ID is already assigned to the registered employee
7	Duplicate record	Save the same employee and date again	Warning: record exists; use Update Attendance to edit
8	Valid new record	Fill all fields and press Save	Record stored, new employee registered, form cleared, all reports refreshed
9	Update	Right-click a row, choose Update Attendance	Row loaded into the form; saving runs UPDATE and refreshes the reports
10	Delete	Right-click a row, choose Delete Attendance	Confirmation dialog, then the row is removed
11	Invalid week or month	Enter a bad date or month	Warning with the expected format
12	No data for a report	Search a date with no records	Message: No attendance records found for that date

No.	Scenario	Action	Behaviour implemented
13	Export with no data	Press Export CSV on an empty table	Warning: Generate a report before exporting it.
14	Test data	Press Generate Test Data	Confirmation, then only missing weekday records are inserted

Summary of Results

The application records, edits and analyses attendance correctly. For the sample data it reports 6 employees, 258 records, 241 present, 17 absent and an overall attendance of 93.4%. The ranking identifies Priya Nair (ID 4) with the highest attendance at 95.3% and Kavya Rao (ID 6) with the lowest at 90.7%. The date-wise search for 2026-09-08 lists one absent employee together with the reason, and the weekly and monthly reports show present days, absent days, totals and percentages for each employee.

Interpretation

Because every report is produced by aggregate queries on the same table, the figures on the dashboard, weekly report and monthly report are always consistent with each other. The percentage depends on the number of records in the chosen period, so a short period such as one week can show large swings (one absence in five days is 80.0%), while a longer period gives a steadier picture. Only weekdays are generated by the test-data routine, so holidays and weekends are not counted as absences.

Achievement of Objectives

The project met its objectives: attendance can be saved, updated and deleted under strict validation; the dashboard, date-wise, weekly, monthly and ranking reports are available with CSV export; and the system runs as one standard-library Python file with a self-upgrading SQLite database.

Implications

The tool can be used by small offices, laboratories, classrooms and clubs that need dependable attendance records without a server or subscription. It is also a useful teaching example of GUI programming, SQL aggregation, validation and database migration in Python.

6 Mapping POs ,SDGs and PSOs

6.1 Program Outcomes (POs) Mapping

Table 5: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied programming, database and data-structure concepts to build a working attendance management system in Python.
PO2	Problem Analysis	Analysed the weaknesses of manual attendance registers (duplicates, missing reasons, slow totals) and derived validation and reporting requirements.
PO3	Design/Development of Solutions	Designed a three-layer desktop application with a normalised SQLite schema, a validated entry form and multiple report screens.
PO4	Conduct Investigations of Complex Problems	Tested validation rules and compared daily, weekly, monthly and overall results on generated data to confirm that the calculations agree.
PO5	Engineering Tool Usage	Used Python, Tkinter/ttk, sqlite3, csv and datetime, together with SQL aggregation queries, to implement and visualise the reports.
PO8	Individual and Collaborative Team work	Worked independently on coding and documentation while collaborating for reviews, improvements and testing.
PO9	Communication	Presented results through dashboard cards, tables, CSV exports and this report, and explained the design and findings clearly.
PO11	Life-Long Learning	Learned GUI event handling, schema migration and parameterised SQL beyond the classroom through self-study and practice.

6.2 Sustainable Development Goals (SDGs) Mapping

Table 6: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 8	Decent Work and Economic Growth	Accurate, transparent attendance records support fair payroll and leave management and build practical skills for employment in IT.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital tools and software innovation by replacing paper registers with a validated, searchable database application.
SDG 12	Responsible Consumption and Production	Reduces paper use by keeping attendance and reports in digital form with CSV export instead of printed registers.

6.3 Program Specific Outcomes (PSOs) Mapping

PSO No.	Program Specific Outcome	Relevance
PSO1	Professional Skills: ability to analyse, design and develop computer-based systems using programming languages, databases and software tools.	Designed and built the Employee Attendance Analyzer in Python with a Tkinter interface and a normalised SQLite database, in three layers (interface, logic, data).
PSO2	Problem-Solving Skills: ability to apply standard software engineering practices and strategies to develop quality software.	Applied input validation, duplicate-record protection, parameterised SQL, safe database migration and SQL aggregation to produce accurate daily, weekly, monthly and ranking reports.
PSO3	Successful Career and Entrepreneurship: ability to use knowledge of different domains to provide innovative, practical solutions.	Delivered a ready-to-use, single-file tool with CSV export that small offices and institutions can adopt without servers or subscriptions. It can be extended with login, charts and biometric check-in.

Conclusion

This project successfully implemented the Employee Attendance Analyzer, a Python desktop application that records employee attendance in an SQLite database and analyses it through a dashboard, date-wise search, weekly and monthly reports and an attendance ranking. Strict validation, duplicate protection, update and delete options, automatic migration of older databases and CSV export make the system dependable and practical for everyday use, while the single-file, standard-library design keeps it simple to install and maintain.

For future enhancements, the application can be extended with administrator login and user roles, leave types and half-day attendance, a holiday calendar so that non-working days are excluded, shift and time-in/time-out tracking, charts for attendance trends, PDF reports, automatic database backup, and QR-code or biometric check-in. A web or mobile front end connected to a shared database would allow several departments to use the system at the same time.

Bibliography

Online Resources

[1] Python Software Foundation – tkinter: Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html

[2] Python Software Foundation – tkinter.ttk: Tk themed widgets (Treeview, Combobox). https://docs.python.org/3/library/tkinter.ttk.html

[3] Python Software Foundation – sqlite3: DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html

[4] Python Software Foundation – csv: CSV file reading and writing. https://docs.python.org/3/library/csv.html

[5] Python Software Foundation – datetime: Basic date and time types. https://docs.python.org/3/library/datetime.html

[6] SQLite Documentation. https://www.sqlite.org/docs.html

[7] TkDocs – Modern Tk Tutorial for Python. https://tkdocs.com/

Research article references

[1] Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.

[2] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.

[3] Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

[4] Grayson, J. E. (2000). Python and Tkinter Programming. Manning Publications.